

AI wrote the query.

It looked right. It was wrong.

What every data professional still needs to own in
2026

• • • query.sql

```
SELECT customer_id, SUM(order_total)  
FROM orders JOIN order_items USING (order_id)  
GROUP BY 1; █
```

looks plausible...

JOINS & Table Grain

orders	
order_id	447
total	\$120
1 row per order	



order_items	
447 · item A	
447 · item B	
447 · item C	
447 · item D	

joined result	
SUM(order_total)	\$480 ✗

AI generates this silently. You catch it — or your metrics are wrong.

1 order × 4 line items = SUM inflated 4×

Window Functions

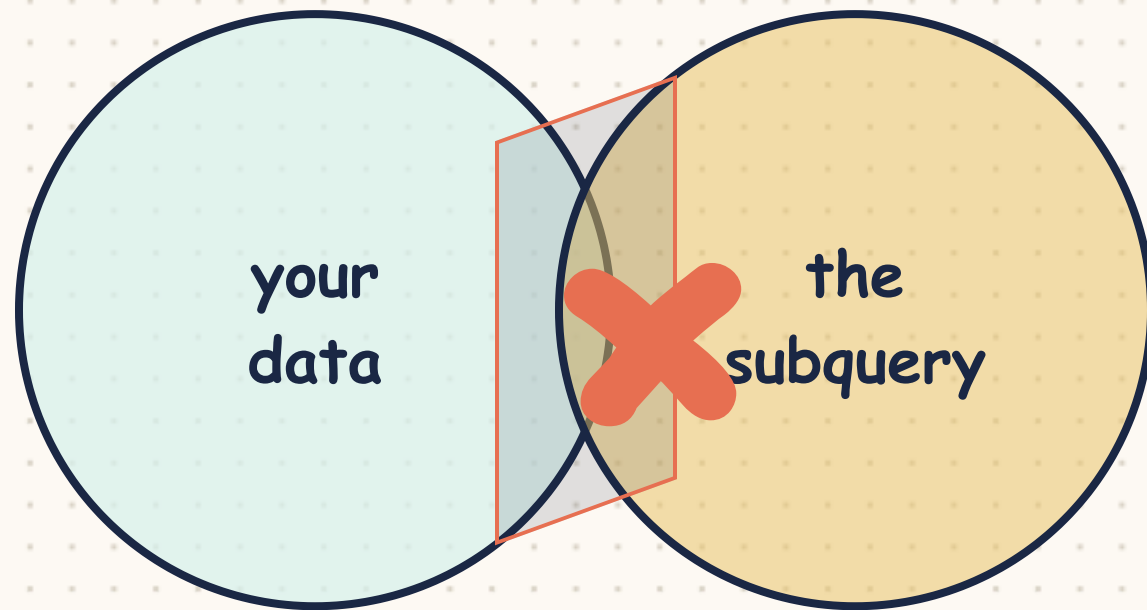
PARTITION: East	PARTITION: West	Functions
Amira 1	Ivo 1	ROW_NUMBER()
Jae 2	Nia 2	RANK()
Mira 3	Sol 3	LAG()

PARTITION BY groups the rows · **ORDER BY** sequences them *inside OVER()*

Placement inside OVER() = most common AI mistake

ROW_NUMBER · RANK · DENSE_RANK · LAG · LEAD · Running SUM

NULL Semantics



NOT IN (subquery with NULL)

Returns zero rows. No error.

NOT EXISTS

Safe alternative ✓

GROUP BY Correctness

WHERE

filters rows **BEFORE** aggregation

rows →

filter →

SUM

HAVING

filters groups **AFTER** aggregation

rows →

SUM →

filter

One character of
difference.
Completely different result.

WHERE revenue > 100

vs

HAVING SUM(revenue) > 100

CTEs over Nested Subqueries

Step 1: clean



Step 2: aggregate



Final: join

CTE chain ✓

```
SELECT (...  
  (SELECT (...  
    (SELECT  
      ...)))
```

nested subquery hell ✗

Multi-step logic. Readable. Auditable.
Debuggable.

WITH clause

Recursive CTEs

Chained logic

Indexing & Sargability



```
WHERE LEFT(name, 1)
= 'A'
```

full table scan



```
WHERE name
LIKE 'A%'
```

index used

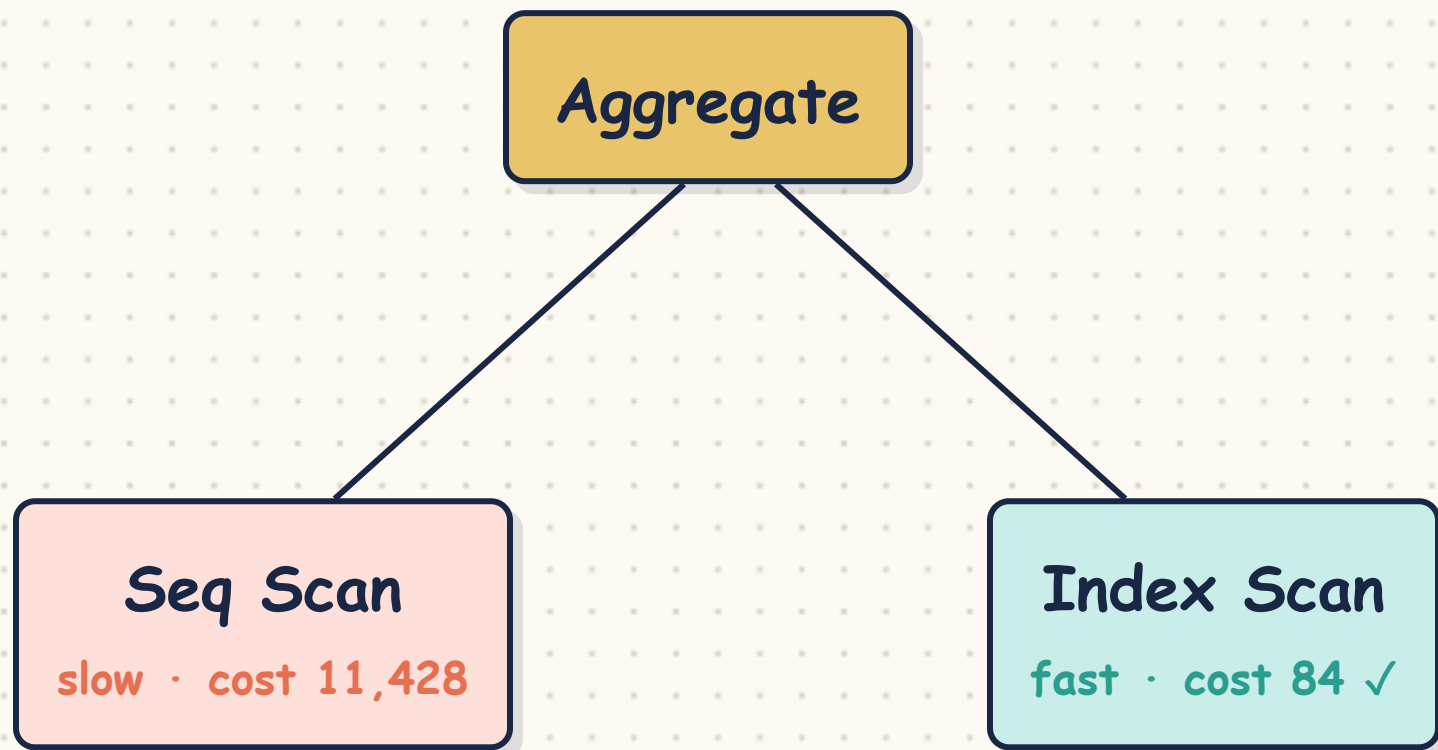
Wrapping a column in a function **kills the index.**

B-Tree

Partition pruning

Clustered vs non-clustered

Read Your EXPLAIN Plan



AI can't run **EXPLAIN ANALYZE** on your data. You must.

Snowflake · BigQuery · Postgres — all expose this

AI vs. You

AI handles

- ◆ date syntax variants
- ◆ dialect differences
- ◆ window frame boilerplate
- ◆ string function names

You must own

- ◆ JOIN cardinality
- ◆ NULL semantics
- ◆ execution plans
- ◆ business context
- ◆ table grain
- ◆ schema design

The skill shifted from writer to reviewer.
Reviewing requires **MORE** understanding.

SQL isn't about memorizing syntax.

It's about knowing what
the data is doing.

Write less.

Review more.

Understand everything.

Which of these still trips you up?
Drop it in the comments 🙌